Documentation Generation -Automatic documentation and code comment s

NAME: J.Shyamsundar

ROLL.NO:2303A52384

PROBLEM-1

Consider the following Python function:

def find_max (numbers):

return max(numbers)

Task:

Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google- style documentation

Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.

Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

PROMPT:

Generate documentation for the find_max (numbers) Python function in three formats: simple docstring, inline comments, and Google-style documentation. Also compare their advantages, disadvantages, and suitable use cases, and recommend the best style for a mathematical utilities library.

INPUT CODE:

(A) **Docstring**

```
def find_max (numbers):
    """Return the maximum value from a list of
numbers."""
    return max(numbers)



numbers = [10, 25, 7, 40, 15]
print("Maximum value:",  find_max (numbers))
```

(B) **Inline comments**

```
def find_max (numbers):
    # Find and return the largest number in the list
    return max(numbers)



numbers = [10, 25, 7, 40, 15]
print("Maximum value:",  find_max (numbers))
```

(C) **Google- style documentation**

```
def find_max (numbers):
    """Find the maximum value in a list of numbers.

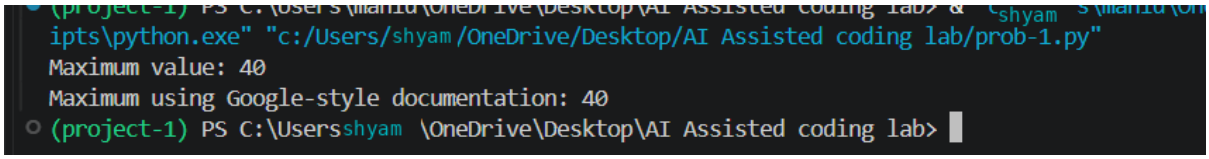
    Args :
        numbers (list): A list of numeric values.

    Returns:
        int or float: The largest value in the list.
```

```
    """
    return max(numbers)


numbers = [10, 25, 7, 40, 15]
print("Maximum value:",  find_max (numbers))
```

OUTPUT:



COMPARISION TABLE:

Style	Advantages	Disadvantages
DOC-STRING	Simple, readable, available through help()	Less detailed
INLINE-COMMENTS	Easy to understand individual steps	Can become outdated and does not appear in documentation tools
Google-style	Detailed, structured, professional	Takes more time to write

Problem 2: Consider the following Python function:

```
def login(user, password, credentials):
```

return credentials.get (user) == password

Task:

1. Write documentation in all three formats.

2. Critically compare the approaches.

3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice .

PROMPT:

Generate documentation for the login(user, password, credentials) Python function in three formats: simple docstring, inline comments, and Google-style documentation. Compare the three approaches and recommend the best style for new developers onboarding a project.

INPUT CODE:

(1) Docstring

```
def login(user, password, credentials):
    """Check whether the username and password are valid."""
    return credentials.get (user) == password



credentials = {
    "admin": "1234",
    "student": " abcd "
}


print(login("admin", "1234", credentials))
```

(2) Inline comments

```
def login(user, password, credentials):
    # Get the stored password for the user
    # Compare it with the password provided
    return  credentials.get (user) == password



credentials = {
    "admin": "1234",
    "student": " abcd "
}

print(login("admin", "1234", credentials))
```

(3) Google- style documentation

```
def login(user, password, credentials):
    """Validate a user's login credentials.

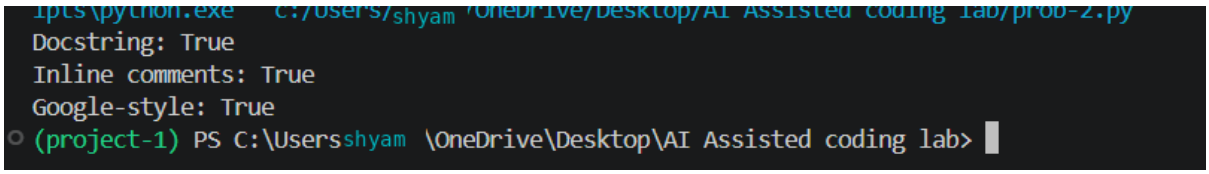
    Args :
        user (str): Username entered by the user.
        password (str): Password entered by the user.
        credentials ( dict ): Dictionary containing usernames and
passwords.

    Returns:
        bool: True if credentials are valid, otherwise False.
    """
    return  credentials.get (user) == password
```

```
credentials = {
    "admin": "1234",
    "student": " abcd "
}

print(login("admin", "1234", credentials))
```

OUTPUT:



COMPARISION TABLE:

Aspect	Docstring	Inline Comments	Google-Style Documentation
Purpose	Gives a short description of the function	Explains specific steps or logic	Gives detailed and structured information
Advantages	Easy to write, keeps code clean, available through help()	Easy to understand program flow and logic	Clearly explains parameters, return values, and purpose

Disadvantages	Less detailed	Can become outdated and is less structured	Takes more time to write
Readability	Good	Good for understanding individual steps	Very good
Suitable for	Small/ simple functions	Complex logic or specific code steps	Large projects, libraries, and team projects
Use in onboarding	Moderate	Limited	Excellent
Recommendation	Good for simple functions	Good as additional explanation	**Best for new developers**

PROBLEM-3

Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.

3. Generate and export the module documentation in HTML

format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

PROMPT:

Create a calculator.py module with add, subtract, multiply, and divide functions. Add clear Google-style docstrings to each function and show how to generate terminal and HTML documentation using Python pydoc.

INPUT CODE:

```
"""Calculator module providing basic mathematical operations."""


def add(a, b):
    """Return the sum of two numbers.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Sum of a and b.
```

```
    """
    return a + b


def subtract(a, b):
    """Return the difference between two numbers.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Difference between a and b.
    """
    return a - b


def multiply(a, b):
    """Return the product of two numbers.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Product of a and b.
```

```
    """
    return a * b


def divide(a, b):
    """Return the quotient of two numbers.

    Args:
        a (int or float): Dividend.
        b (int or float): Divisor.

    Returns:
        float: Quotient of a and b.

    Raises:
        ZeroDivisionError: If b is zero.
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b
import calculator

print("Addition:", calculator.add(10, 5))
print("Subtraction:", calculator.subtract(10, 5))
print("Multiplication:", calculator.multiply(10, 5))
print("Division:", calculator.divide(10, 5))
```

OUTPUT:

```
ipts\python.exe" "c:/Users/shyam/OneDrive/Desktop/AI Assisted coding lab/prob-3.py"
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
```

index

calculator c:\users\shyam onedrive\desktop\ai assisted coding lab\calculator.py

Conversion utilities for decimal, binary, and hexadecimal values.

Functions

binary_to_decimal(b)
Convert a binary number to decimal.

Args:
b (str): Binary number.

Returns:
int: Decimal representation of the binary number.

decimal_to_binary(n)
Convert a decimal number to binary.

Args:
n (int): Decimal number.

Returns:
str: Binary representation of the number.

decimal_to_hexadecimal(n)
Convert a decimal number to hexadecimal.

Args:
n (int): Decimal number.

Returns:
str: Hexadecimal representation of the number.

PROBLEM-4:

Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary (n)

o binary_to_decimal (b)

o decimal_to_hexadecimal (n)

2. Use Copilot for auto- generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

PROMPT:

Create a conversion.py module with decimal_to_binary , binary_to_decimal , and decimal_to_hexadecimal functions. Generate clear Google-style docstrings for all functions and explain how to generate terminal and HTML documentation using pydoc .

INPUT CODE:

```
"""Conversion utilities for decimal, binary, and hexadecimal values."""



def decimal_to_binary (n):
    """Convert a decimal number to binary.

    Args :
        n (int): Decimal integer.

    Returns:
```

```
        str: Binary representation of the number.
    """
    return bin(n)[2:]


def binary_to_decimal(b):
    """Convert a binary number to decimal.

    Args:
        b (str): Binary number as a string.

    Returns:
        int: Decimal representation of the binary number.
    """
    return int(b, 2)


def decimal_to_hexadecimal(n):
    """Convert a decimal number to hexadecimal.

    Args:
        n (int): Decimal integer.

    Returns:
        str: Hexadecimal representation of the number.
    """
```

```
    return hex(n)[2:]


if __name__ == "__main__":
    print("Decimal to Binary:", decimal_to_binary(10))
    print("Binary to Decimal:", binary_to_decimal("1010"))
    print("Decimal to Hexadecimal:",
decimal_to_hexadecimal(255))
```

OUTPUT:

```
Ipts\python.exe c:/Users/shyam/OneDrive/Desktop/AI Assisted Coding lab/prob-4.py
Decimal to Binary: 1010
Binary to Decimal: 10
Decimal to Hexadecimal: ff
```

```
NAME
    CONVERSION - Conversion utilities for decimal, binary, and hexadecimal values.

FUNCTIONS
    binary_to_decimal(b)
        Convert a binary number to decimal.

        Args:
            b (str): Binary number as a string.
```

index

CONVERSION c:\users\shyam onedrive\desktop\ai assisted coding lab\conversion.py

Conversion utilities for decimal, binary, and hexadecimal values.

Functions

binary_to_decimal(b)
Convert a binary number to decimal.

Args:
b (str): Binary number as a string.

Returns:
int: Decimal representation of the binary number.

decimal_to_binary(n)
Convert a decimal number to binary.

Args:
n (int): Decimal integer.

Returns:
str: Binary representation of the number.

decimal_to_hexadecimal(n)
Convert a decimal number to hexadecimal.

Args:
n (int): Decimal integer.

Returns:
str: Hexadecimal representation of the number.

PROBLEM-5:

Course Management Module

Task:

1. Create a module course.py with functions:

o add_course (course_id , name, credits)

o remove_course (course_id)

o get_course (course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

PROMPT:

Create a course.py module with add_course , remove_course , and get_course functions. Add Google-style docstrings using Copilot and show how to generate terminal and HTML documentation using Python pydoc .

INPUT CODE:

```
"""Course management module for adding, removing, and retrieving courses."""

courses = {}


def add_course (course_id , name, credits):
    """Add a new course to the course collection.

    Args :
        course_id (str): Unique ID of the course.
        name (str): Name of the course.
        credits (int): Number of credits for the course.

    Returns:
        str: Confirmation message.
    """
```

```
    courses[ course_id ] = {
        "name": name,
        "credits": credits
    }
    return "Course added successfully"


def remove_course (course_id ):
    """Remove a course using its course ID.

    Args :
        course_id  (str): ID of the course to remove.

    Returns:
        str: Confirmation or error message.
    """
    if course_id  in courses:
        del courses[ course_id ]
        return "Course removed successfully"
    return "Course not found"


def get_course (course_id ):
    """Retrieve course details using the course ID.

    Args :
```

```
        course_id (str): ID of the course to retrieve.

    Returns:
        dict or None: Course details if found, otherwise None.
    """
    return courses.get(course_id)

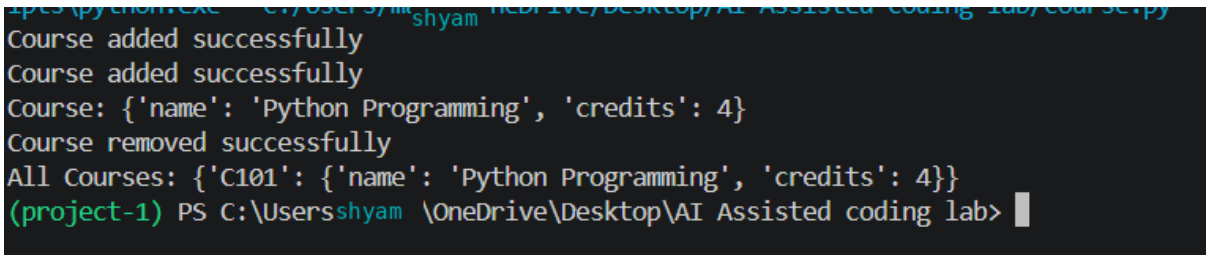

if __name__ == "__main__":
    print(add_course("C101", "Python Programming", 4))
    print(add_course("C102", "Database Management", 3))

    print("Course:", get_course("C101"))

    print(remove_course("C102"))

    print("All Courses:", courses)
```

OUTPUT:



```
(project-1) PS C:\Users\shyam \OneDrive\Desktop\AI Assisted coding lab> python -m pydoc course

NAME
    course - Course management module for adding, removing, and retrieving courses.

FUNCTIONS
    add_course(course_id, name, credits)
        Add a new course to the course collection.

        Args:
-- More  --
```

Course management module for adding, removing, and retrieving courses.

Functions

add_course(course_id, name, credits)
Add a new course to the course collection.

Args:
course_id (str): Unique ID of the course.
name (str): Name of the course.
credits (int): Number of credits for the course.

Returns:
str: Confirmation message.

get_course(course_id)
Retrieve course details using the course ID.

Args:
course_id (str): ID of the course to retrieve.

Returns:
dict or None: Course details if found, otherwise None.

remove_course(course_id)
Remove a course using its course ID.

Args:
course_id (str): ID of the course to remove.

Returns:
str: Confirmation or error message.

Data

courses = {}